PROJECT REPORT

CS-220

DATABASES PROJECT REPORT

INSTRUCTOR: Maryam Sajjad

Student Name	CMS ID
Affan Tariq	551279
Ali Hassan	549075
Sibghatullah Saeed	542378

CS -15 Section A

Date Submitted: April 23, 2026

School of Electrical Engineering and Computer Science

National University of Science and Technology

Table of Contents

Table of Contents 2
1. Introduction 4
3. Entity–Relationship Diagram 5
4. Relational Schema 6
4.1 GAME 6
4.2 PLAYER 6
4.3 TEAM 7
4.4 ROSTER 7
4.5 TOURNAMENT 8
4.6 TOURNAMENT_REGISTRATION 8
4.7 MATCH 9
4.8 PRIZE_DISTRIBUTION 9
4.9 SPONSOR 10
4.10 SPONSORSHIP 10
5.1 3NF Normalization Violations Fixed 13
5.2 DDL Script 13
6. SQL DML — Sample Data 18
7.1 Data Insertion Queries 22
7.1.1 Insert a New Team 22
7.1.2 Insert a New Player 22
7.1.3 Register a Team in a Tournament 22
7.2 Data Update Queries 22
7.2.1 Update Team Region 22
7.2.2 Update Prize Amount 22
7.3 Data Deletion Queries 22
7.3.1 Delete a Player 22
7.3.2 Remove Tournament Registration 23
7.4 Analytical Queries (Reports) 23
7.4.1 Total Earnings of Each Team 23
7.4.2 Top 3 Performing Teams 23
7.4.3 Tournament with Highest Prize Pool 23
7.4.4 Total Sponsorship per Tournament 23
7.4.5 Player Earnings Leaderboard 23

7.4.6 Teams with No Earnings 24
7.4.7 Most Popular Game (by Number of Tournaments) 24
8. Database Optimization Report 24
8.1 Indexing Strategy 24
8.2 Triggers for Constraint Enforcement 24
8.3 Views for Derived Data 25
8.4 Normalization Benefits 25
9. Conclusion 25
References 26

1. Introduction

NexusArena is a fully normalized relational database system designed to manage the operations of a professional esports organization. The system tracks players, teams, games, tournaments, match results, prize distributions, and commercial sponsorships. It was implemented in MySQL 8.x and normalized to Third Normal Form (3NF) to eliminate redundancy and ensure long-term data integrity.

The project fulfils the requirements of CS220 — Database Systems, demonstrating the complete lifecycle of database development: requirements analysis, entity–relationship modelling, relational schema derivation, DDL implementation, sample data population, and performance optimization.

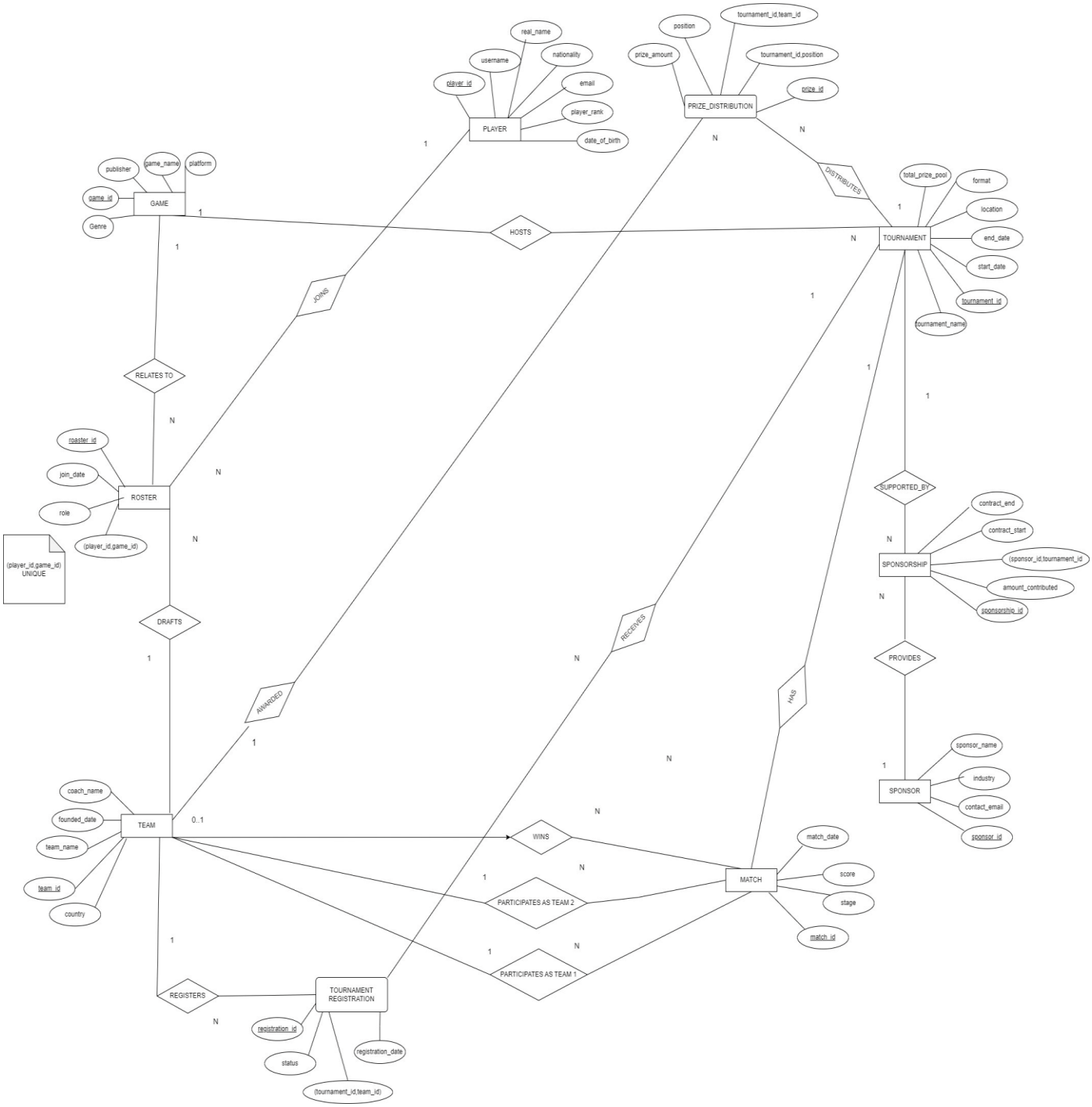
2. Project Scenario

The esports industry has grown into a multi-billion-dollar sector requiring sophisticated data management across tournaments, players, sponsors, and competitive results. NexusArena models a platform that:

- Registers and tracks professional esports players and teams across multiple game titles.
- Manages tournament schedules, registrations, match brackets, and stage-by-stage results.
- Records prize distributions per tournament position, replacing error-prone derived columns.
- Maintains commercial sponsorship contracts between corporate sponsors and tournaments.
- Exposes computed analytics (earnings, leaderboards) through SQL views rather than redundant stored columns.

3. Entity–Relationship Diagram

The ER diagram below was designed in draw.io and represents all entities, attributes, and relationships in the NexusArena schema. Cardinality notations follow standard crow's-foot / Chen notation as taught in CS220.



4. Relational Schema

The relational schema below is derived directly from the ER diagram and exactly matches the DDL implemented in Section 5. Primary keys are underlined; foreign keys are marked with (FK). Attributes removed during 3NF normalization are noted in parentheses.

4.1 GAME

GAME (game_id, game_name, publisher, platform, genre)

Attribute	Type	Constraints
game_id	INT	PRIMARY KEY, AUTO_INCREMENT
game_name	VARCHAR(100)	NOT NULL
publisher	VARCHAR(100)	NOT NULL
platform	VARCHAR(50)	
genre	VARCHAR(50)	

4.2 PLAYER

“total_earnings” removed — 3NF violation (transitive dependency via ROSTER).

PLAYER (player_id, username, real_name, nationality, email, player_rank, date_of_birth)

Attribute	Type	Constraints
player_id	INT	PRIMARY KEY, AUTO INCREMENT
username	VARCHAR(50)	NOT NULL, UNIQUE
real_name	VARCHAR(100)	NOT NULL
nationality	VARCHAR(50)	
email	VARCHAR(100)	UNIQUE
player_rank	VARCHAR(30)	
date_of_birth	DATE	

4.3 TEAM

"total_prize_money" removeddue to 3NF violation (transitive dependency). Computed via vw_team_earnings.

TEAM (team_id, team_name, founded_date, coach_name, country)

Attribute	Type	Constraints
team_id	INT	PRIMARY KEY, AUTO_INCREMENT
team_name	VARCHAR(100)	NOT NULL, UNIQUE
founded_date	DATE	
coach_name	VARCHAR(100)	
country	VARCHAR(50)	

4.4 ROSTER

Bridge table resolving M:N between PLAYER, TEAM, and GAME. UNIQUE(player_id, game_id) enforces the rule that a player may only be on one team per game.

ROSTER (roster_id, player_id (FK), team_id (FK), game_id (FK), join_date, role)

Attribute	Type	Constraints
roster_id	INT	PRIMARY KEY, AUTO_INCREMENT
player_id	INT	NOT NULL, FK → PLAYER(player_id) CASCADE
team_id	INT	NOT NULL, FK → TEAM(team_id) CASCADE
game_id	INT	NOT NULL, FK → GAME(game_id) CASCADE
join_date	DATE	
role	VARCHAR(50)	
—	UNIQUE	(player_id, game_id)

4.5 TOURNAMENT

TOURNAMENT (tournament_id, tournament_name, game_id (FK), start_date, end_date, location, format, total_prize_pool)

Attribute	Type	Constraints
tournament_id	INT	PRIMARY KEY, AUTO_INCREMENT
tournament_name	VARCHAR(150)	NOT NULL
game_id	INT	NOT NULL, FK → GAME(game_id) RESTRICT
start_date	DATE	NOT NULL
end_date	DATE	NOT NULL, CHECK >= start_date
location	VARCHAR(100)	NOT NULL
format	VARCHAR(60)	NOT NULL
total_prize_pool	DECIMAL(12,2)	NOT NULL

4.6 TOURNAMENT_REGISTRATION

TOURNAMENT_REGISTRATION (registration_id, tournament_id (FK), team_id (FK), registration_date, status)

Attribute	Type	Constraints
registration_id	INT	PRIMARY KEY, AI
tournament_id	INT	NOT NULL, FK → TOURNAMENT CASCADE
team_id	INT	NOT NULL, FK → TEAM
registration_date	DATE	NOT NULL
status	VARCHAR(20)	NOT NULL, DEFAULT 'Pending'
—	UNIQUE	(tournament_id, team_id)

4.7 MATCH

MATCH (match_id, tournament_id (FK), team1_id (FK), team2_id (FK), match_date, stage, winner_team_id (FK), score)

Attribute	Type	Constraints
match_id	INT	PRIMARY KEY, AUTO_INCREMENT
tournament_id	INT	NOT NULL, FK → TOURNAMENT CASCADE
team1_id	INT	NOT NULL, FK → TEAM RESTRICT
team2_id	INT	NOT NULL, FK → TEAM RESTRICT, CHECK != team1_id
match_date	DATETIME	NOT NULL
stage	VARCHAR(50)	NOT NULL
winner_team_id	INT	NULL, FK → TEAM SET NULL (trigger-validated)
score	VARCHAR(20)	NULL

4.8 PRIZE_DISTRIBUTION

PRIZE_DISTRIBUTION (prize_id, tournament_id (FK), team_id (FK), position, prize_amount)

Attribute	Type	Constraints
prize_id	INT	PRIMARY KEY, AUTO_INCREMENT
tournament_id	INT	NOT NULL, FK → TOURNAMENT CASCADE
team_id	INT	NOT NULL, FK → TEAM CASCADE
position	TINYINT	NOT NULL

prize_amount	DECIMAL(12,2)	NOT NULL
—	UNIQUE	(tournament_id, position)
—	UNIQUE	(tournament_id, team_id)

4.9 SPONSOR

SPONSOR (sponsor_id, sponsor_name, industry, contact_email)

Attribute	Type	Constraints
sponsor_id	INT	PRIMARY KEY, AUTO_INCREMENT
sponsor_name	VARCHAR(100)	NOT NULL, UNIQUE
industry	VARCHAR(80)	NOT NULL
contact_email	VARCHAR(100)	NOT NULL

4.10 SPONSORSHIP

SPONSORSHIP (sponsorship_id, sponsor_id (FK), tournament_id (FK), amount_contributed, contract_start, contract_end)

Attribute	Type	Constraints
sponsorship_id	INT	PRIMARY KEY, AUTO_INCREMENT
sponsor_id	INT	NOT NULL, FK → SPONSOR
tournament_id	INT	NOT NULL, FK → TOURNAMENT CASCADE
amount_contributed	DECIMAL(12,2)	NOT NULL
contract_start	DATE	NOT NULL
contract_end	DATE	NOT NULL, CHECK >= contract_start
—	UNIQUE	(sponsor_id, tournament_id)

4.11 RELATIONAL SCHEMA:

4.11.1 GAME

GAME(<u>game_id</u>, game_name, publisher, platform, genre)

4.11.2 PLAYER

PLAYER(<u>player_id</u>, username, real_name, nationality, email, player_rank, date_of_birth)

4.11.3 TEAM

TEAM(<u>team_id</u>, team_name, founded_date, coach_name, country)

4.11.4 ROSTER

ROSTER(<u>roster_id</u>, player_id, team_id, game_id, join_date, role)
FK: player_id → PLAYER(player_id)
FK: team_id → TEAM(team_id)
FK: game_id → GAME(game_id)

4.11.5 TOURNAMENT

TOURNAMENT(<u>tournament_id</u>, tournament_name, game_id, start_date, end_date, location, format, total_prize_pool)
FK: game_id → GAME(game_id)

4.11.6 TOURNAMENT_REGISTRATION

TOURNAMENT_REGISTRATION(<u>registration_id</u>, tournament_id, team_id, registration_date, status)
FK: tournament_id → TOURNAMENT(tournament_id)
FK: team_id → TEAM(team_id)

4.11.7 MATCH

MATCH(<u>match_id</u>, tournament_id, team1_id, team2_id, winner_team_id, match_date, stage, score)
FK: tournament_id → TOURNAMENT(tournament_id)
FK: team1_id → TEAM(team_id)
FK: team2_id → TEAM(team_id)
FK: winner_team_id → TEAM(team_id)

4.11.8 PRIZE_DISTRIBUTION

PRIZE_DISTRIBUTION(<u>prize_id</u>, tournament_id, team_id, position, prize_amount)
FK: tournament_id → TOURNAMENT(tournament_id)
FK: team_id → TEAM(team_id)

4.11.9 SPONSOR

SPONSOR(<u>sponsor_id</u>, sponsor_name, industry, contact_email)

4.11.10 SPONSORSHIP

SPONSORSHIP(<u>sponsorship_id</u>, sponsor_id, tournament_id, amount_contributed, contract_start, contract_end)
FK: sponsor_id → SPONSOR(sponsor_id)
FK: tournament_id → TOURNAMENT(tournament_id)

5. SQL DDL — Data Definition Language

The following scripts create the NexusArena database, all tables, constraints, indexes, triggers, and views. The schema has been normalized to Third Normal Form (3NF). Six corrections were applied during normalization; each is annotated inline.

5.1 3NF Normalization Violations Fixed

Fix	Violation	Resolution
1	PLAYER.total_earnings — transitive dependency	Column removed; computed via view vw_player_earnings
2	TEAM.total_prize_money — transitive dependency	Column removed; computed via view vw_team_earnings
3	ROSTER UNIQUE was (player_id, team_id, game_id)	Changed to UNIQUE(player_id, game_id) — one team per game
4	PRIZE_DISTRIBUTION missing UNIQUE(tournament_id, team_id)	Added UNIQUE(tournament_id, team_id)
5	SPONSORSHIP missing CHECK(contract_end >= contract_start)	CHECK constraint added
6	Anonymous foreign keys	All FKs renamed with explicit constraint names

5.2 DDL Script

```
CREATE DATABASE NexusArena;
USE NexusArena;
-- TABLE: GAME
CREATE TABLE GAME (
    game_id INT PRIMARY KEY AUTO_INCREMENT,
    game_name VARCHAR(100) NOT NULL,
    publisher VARCHAR(100) NOT NULL,
    platform VARCHAR(50),
    genre VARCHAR(50));
-- TABLE: PLAYER
-- FIX:total_earnings removed(transitive dependency – 3NF violation).
-- (Derived Data)Computed on demand via vw_player_earnings.
CREATE TABLE PLAYER (
    player_id INT PRIMARY KEY AUTO_INCREMENT,
    username VARCHAR(50)  NOT NULL UNIQUE,
```

```
    real_name VARCHAR(100) NOT NULL,
    nationality VARCHAR(50),
    email VARCHAR(100) UNIQUE,
    player_rank VARCHAR(30),
    date_of_birth DATE);
-- TABLE: TEAM
-- FIX 2: total_prize_money removed (transitive dependency — 3NF violation).
-- (Derived Data)Computed on demand via vw_team_earnings.
CREATE TABLE TEAM (
    team_id INT PRIMARY KEY AUTO_INCREMENT,
    team_name VARCHAR(100) NOT NULL UNIQUE,
    founded_date DATE,
    coach_name VARCHAR(100),
    country VARCHAR(50));


-- TABLE: ROSTER
-- FIX 3: UNIQUE changed from (player_id, team_id, game_id)
--        to (player_id, game_id) — a player may only be on
--        ONE team per game.
CREATE TABLE ROSTER (
    roster_id INT PRIMARY KEY AUTO_INCREMENT,
    player_id INT NOT NULL,
    team_id   INT NOT NULL,
    game_id   INT NOT NULL,
    join_date DATE,
    role      VARCHAR(50),
    UNIQUE (player_id, game_id),
    CONSTRAINT fk_roster_player FOREIGN KEY (player_id) REFERENCES PLAYER(player_id) ON
DELETE CASCADE,
    CONSTRAINT fk_roster_team   FOREIGN KEY (team_id)   REFERENCES TEAM(team_id)     ON
DELETE CASCADE,
    CONSTRAINT fk_roster_game   FOREIGN KEY (game_id)   REFERENCES GAME(game_id)     ON
DELETE CASCADE);


-- TABLE: TOURNAMENT
-- CHECK ensures end_date >= start_date.Vice Versa Can never Be true.
CREATE TABLE TOURNAMENT (
    tournament_id    INT           PRIMARY KEY AUTO_INCREMENT,
    tournament_name  VARCHAR(150)  NOT NULL,
    game_id          INT           NOT NULL,
    start_date       DATE          NOT NULL,
    end_date         DATE          NOT NULL,
    location         VARCHAR(100)  NOT NULL,
    format           VARCHAR(60)   NOT NULL,
    total_prize_pool DECIMAL(12,2) NOT NULL,--Using 12,2 Since the value here can be large
    CONSTRAINT chk_tournament_dates CHECK (end_date >= start_date),
```

```
CONSTRAINT fk_tournament_game   FOREIGN KEY (game_id) REFERENCES GAME(game_id) ON DELETE
RESTRICT
);
-- TABLE: TOURNAMENT_REGISTRATION
-- UNIQUE(tournament_id, team_id) prevents duplicate registrations.
CREATE TABLE TOURNAMENT_REGISTRATION (
    registration_id INT PRIMARY KEY AUTO_INCREMENT,
    tournament_id INT NOT NULL,
    team_id INT NOT NULL,
    registration_date DATE NOT NULL,
    status  VARCHAR(20) NOT NULL DEFAULT 'Pending',
    UNIQUE (tournament_id, team_id),
    CONSTRAINT fk_reg_tournament FOREIGN KEY (tournament_id) REFERENCES
TOURNAMENT(tournament_id) ON DELETE CASCADE,
    CONSTRAINT fk_reg_team FOREIGN KEY (team_id) REFERENCES TEAM(team_id) ON DELETE CASCADE);

-- TABLE: MATCH  ( Delimiited Variable)
-- CHECK prevents a team from facing itself.
-- Winner validation enforced via TRIGGERS (MySQL limitation).
CREATE TABLE `MATCH` (
    match_id INT PRIMARY KEY AUTO_INCREMENT,
    tournament_id  INT NOT NULL,
    team1_id INT NOT NULL,
    team2_id INT NOT NULL,
    match_date DATETIME NOT NULL,
    stage VARCHAR(50) NOT NULL,
    winner_team_id INT  NULL,
    score VARCHAR(20) NULL,
    CONSTRAINT chk_different_teams CHECK (team1_id != team2_id),
    CONSTRAINT fk_match_tournament FOREIGN KEY (tournament_id)  REFERENCES
TOURNAMENT(tournament_id) ON DELETE CASCADE,
    CONSTRAINT fk_match_team1 FOREIGN KEY (team1_id) REFERENCES TEAM(team_id) ON DELETE
RESTRICT,
    CONSTRAINT fk_match_team2 FOREIGN KEY (team2_id) REFERENCES TEAM(team_id) ON DELETE
RESTRICT,
    CONSTRAINT fk_match_winner FOREIGN KEY (winner_team_id) REFERENCES TEAM(team_id)
ON DELETE SET NULL);

-- TABLE: PRIZE_DISTRIBUTION
-- FIX : Added UNIQUE(tournament_id, team_id) in addition to
--        existing UNIQUE(tournament_id, position).
CREATE TABLE PRIZE_DISTRIBUTION (
    prize_id      INT           PRIMARY KEY AUTO_INCREMENT,
    tournament_id INT           NOT NULL,
    team_id       INT           NOT NULL,
    position      TINYINT       NOT NULL,
    prize_amount  DECIMAL(12,2) NOT NULL,
    UNIQUE (tournament_id, position),
```

```
UNIQUE (tournament_id, team_id),
CONSTRAINT fk_prize_tournament FOREIGN KEY (tournament_id) REFERENCES
TOURNAMENT(tournament_id) ON DELETE CASCADE,
CONSTRAINT fk_prize_team FOREIGN KEY (team_id) REFERENCES TEAM(team_id) ON DELETE CASCADE);

-- TABLE: SPONSOR
CREATE TABLE SPONSOR (
    sponsor_id    INT          PRIMARY KEY AUTO_INCREMENT,
    sponsor_name  VARCHAR(100) NOT NULL UNIQUE,
    industry      VARCHAR(80)  NOT NULL,
    contact_email VARCHAR(100) NOT NULL
);
-- TABLE: SPONSORSHIP
-- FIX 5: Added CHECK(contract_end >= contract_start).
-- UNIQUE(sponsor_id, tournament_id) prevents duplicate contracts.

CREATE TABLE SPONSORSHIP (
    sponsorship_id INT PRIMARY KEY AUTO_INCREMENT,
    sponsor_id INT NOT NULL,
    tournament_id INT NOT NULL,
    amount_contributed DECIMAL(12,2) NOT NULL,
    contract_start DATE NOT NULL,
    contract_end DATE NOT NULL,
    UNIQUE (sponsor_id, tournament_id),
    CONSTRAINT chk_contract_dates CHECK (contract_end >= contract_start),
    CONSTRAINT fk_sponsorship_sponsor FOREIGN KEY (sponsor_id)    REFERENCES
SPONSOR(sponsor_id) ON DELETE RESTRICT,
    CONSTRAINT fk_sponsorship_tourn   FOREIGN KEY (tournament_id) REFERENCES
TOURNAMENT(tournament_id) ON DELETE CASCADE);

-- INDEXES
-- For fast searching
CREATE INDEX idx_prize_team       ON PRIZE_DISTRIBUTION(team_id);
CREATE INDEX idx_prize_tournament ON PRIZE_DISTRIBUTION(tournament_id);
CREATE INDEX idx_sponsor_tourn    ON SPONSORSHIP(tournament_id);
-- ============================================================
-- TRIGGERS — Winner Validation for MATCH
-- A new concept learnt for Optimization of databases
-- ============================================================
DELIMITER $$
CREATE TRIGGER trg_match_winner_insert
BEFORE INSERT ON `MATCH`
FOR EACH ROW
BEGIN
    IF NEW.winner_team_id IS NOT NULL AND
       NEW.winner_team_id NOT IN (NEW.team1_id, NEW.team2_id) THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Winner must be one of the two competing teams';
```

```
END IF;
END$$

CREATE TRIGGER trg_match_winner_update
BEFORE UPDATE ON `MATCH`
FOR EACH ROW
BEGIN
    IF NEW.winner_team_id IS NOT NULL AND
       NEW.winner_team_id NOT IN (NEW.team1_id, NEW.team2_id) THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Winner must be one of the two competing teams';
    END IF;
END$$
DELIMITER ;

-- VIEWS(Using Views to compute the Derived data on demand
-- instead of Storing it in a Seprate attribute)

CREATE VIEW vw_player_earnings AS
SELECT p.player_id, p.username, p.real_name,
       COALESCE(SUM(pd.prize_amount), 0) AS total_earnings
FROM PLAYER p
LEFT JOIN ROSTER r             ON p.player_id = r.player_id
LEFT JOIN PRIZE_DISTRIBUTION pd ON r.team_id  = pd.team_id
GROUP BY p.player_id, p.username, p.real_name;

CREATE VIEW vw_team_earnings AS
SELECT t.team_id, t.team_name, t.country,
       COALESCE(SUM(pd.prize_amount), 0) AS total_prize_money
FROM TEAM t
LEFT JOIN PRIZE_DISTRIBUTION pd ON t.team_id = pd.team_id
GROUP BY t.team_id, t.team_name, t.country;

CREATE VIEW vw_global_leaderboard AS
SELECT t.team_name, t.country,
       COALESCE(SUM(pd.prize_amount), 0) AS total_earnings,
       COUNT(pd.prize_id)                AS podium_finishes
FROM TEAM t
LEFT JOIN PRIZE_DISTRIBUTION pd ON t.team_id = pd.team_id
GROUP BY t.team_id, t.team_name, t.country;
```

6. SQL DML — Sample Data

The following INSERT statements populate the NexusArena database with realistic sample data drawn from publicly available esports sources including Liquipedia, HLTV.org, VLR.gg, Esports Charts, and Esports Insider. The dataset covers 8 game titles, 12 professional teams, 15 players, 8 major tournaments, and associated sponsorship data.

```
USE NexusArena;
-- INSERT: GAME


INSERT INTO GAME (game_name, publisher, platform, genre) VALUES
    ('Valorant', 'Riot Games', 'PC', 'Tactical Shooter'),
    ('Counter-Strike 2',  'Valve', 'PC', 'Tactical Shooter'),
    ('League of Legends', 'Riot Games', 'PC', 'MOBA'),
    ('Dota 2', 'Valve', 'PC', 'MOBA'),
    ('Rocket League', 'Psyonix', 'PC/Console', 'Sports'),
    ('Fortnite', 'Epic Games', 'PC/Console', 'Battle Royale'),
    ('Apex Legends', 'Respawn', 'PC/Console', 'Battle Royale'),
    ('Overwatch 2', 'Blizzard', 'PC/Console', 'Team Shooter');

-- INSERT: TEAM

INSERT INTO TEAM (team_name, founded_date, coach_name, country) VALUES
    ('Team Liquid',   '2000-01-01', 'David Holden',   'Netherlands'),
    ('Natus Vincere', '2009-12-17', 'Andrey Bloknot', 'Ukraine'),
    ('Cloud9',        '2012-05-15', 'James Smith',    'USA'),
    ('Fnatic',        '2004-07-23', 'Louis Dobson',   'United Kingdom'),
    ('Astralis',      '2016-01-17', 'Lars Doere',     'Denmark'),
    ('FaZe Clan',     '2016-01-30', 'Mads Lauridsen', 'USA'),
    ('T1',            '2019-09-04', 'Park Jae-hyun',  'South Korea'),
    ('G2 Esports',    '2013-11-25', 'Fabian Lohmann', 'Spain'),
    ('100 Thieves',   '2017-11-06', 'Jordan Terry',   'USA'),
    ('Sentinels',     '2018-01-01', 'Adam Kaplan',    'USA'),
    ('NRG Esports',   '2015-11-11', 'Chet Singh',     'USA'),
    ('Team Vitality', '2013-11-23', 'Nicolas Maurer', 'France');


-- INSERT: PLAYER
INSERT INTO PLAYER (username, real_name, nationality, email, player_rank, date_of_birth)
VALUES
    ('s1mple', 'Oleksandr Kostyliev','Ukrainian','s1mple@nexus.gg','Global Elite', '1997-10-
02'),
    ('ZywOo','Mathieu Herbaut','French','zywoo@nexus.gg','Global Elite', '2000-11-09'),
    ('NiKo','Nikola Kovac','Bosnian','niko@nexus.gg','Global Elite', '1997-02-16'),
    ('device','Nicolai Reedtz','Danish','device@nexus.gg','Global Elite', '1996-09-08'),
    ('Faker',    'Lee Sang-hyeok',      'Korean',    'faker@nexus.gg',    'Challenger',
'1996-05-07'),
```

```
('Caps', 'Rasmus Borregaard', 'Danish', 'caps@nexus.gg', 'Grandmaster', '1999-11-04'),
    ('TenZ', 'Tyson Ngo', 'Canadian',  'tenz@nexus.gg', 'Radiant',      '2001-05-05'),
    ('ScreaM', 'Adil Benrlitom', 'Belgian', 'scream@nexus.gg', 'Radiant', '1994-03-02'),
    ('Yay', 'Jaccob Whitelaw', 'American', 'yay@nexus.gg', 'Radiant', '1999-07-10'),
    ('ShahZaM', 'Shahzeeb Khan', 'American', 'shahzam@nexus.gg', 'Radiant', '1995-06-24'),
    ('Bugha', 'Kyle Giersdorf', 'American', 'bugha@nexus.gg', 'Champion','2002-12-30'),
    ('Aqua', 'David Wang', 'Austrian', 'aqua@nexus.gg', 'Champion',     '2004-03-21'),
    ('Shroud', 'Michael Grzesiek',    'Canadian',  'shroud@nexus.gg',   'Radiant',
'1994-06-02'),
    ('Stewie2K', 'Jacky Yip', 'American', 'stewie2k@nexus.gg', 'Global Elite', '1998-01-07'),
    ('Xyp9x',    'Andreas Hojsleth', 'Danish', 'xyp9x@nexus.gg', 'Global Elite', '1995-09-
11');

-- INSERT: ROSTER

INSERT INTO ROSTER (player_id, team_id, game_id, join_date, role) VALUES
    (1,2,2, '2017-01-01', 'AWPer'),
    (2,12,2, '2019-09-01', 'AWPer'),
    (3,  8,  2, '2021-01-01', 'Entry Fragger'),
    (4,  5,  2, '2016-01-17', 'AWPer'),
    (5,  7,  3, '2013-02-01', 'Mid Laner'),
    (6,  8,  3, '2018-11-22', 'Mid Laner'),
    (7,  10, 1, '2021-02-10', 'Duelist'),
    (8,  8,  1, '2022-01-15', 'Duelist'),
    (9,  9,  1, '2022-06-01', 'Duelist'),
    (10, 10, 1, '2020-09-12', 'IGL'),
    (11, 3,  6, '2019-09-01', 'Solo Carry'),
    (12, 6,  6, '2022-04-01', 'Solo Carry'),
    (13, 3,  2, '2020-03-01', 'Rifler'),
    (14, 3,  2, '2019-07-01', 'Entry Fragger'),
    (15, 5,  2, '2016-01-17', 'Support');

-- INSERT: TOURNAMENT

INSERT INTO TOURNAMENT (tournament_name, game_id, start_date, end_date, location, format,
total_prize_pool) VALUES
    ('VCT Masters Tokyo 2024', 1,'2024-06-02','2024-06-16','Tokyo, Japan', 'Double
Elimination', 500000.00),
    ('ESL Pro League Season 19', 2,'2024-03-20','2024-04-07','Malta', 'Swiss + Playoffs',
835000.00),
    ('IEM Cologne 2024', 2,'2024-07-10','2024-07-21','Cologne, Germany',    'Single
Elimination',1000000.00),
    ('LoL World Championship 2024', 3,'2024-09-25','2024-11-02','London, UK', 'Group +
Knockout',  2250000.00),
    ('Rocket League World Champ 2024', 5,'2024-08-08','2024-08-11','Riyadh, Saudi
Arabia','Double Elimination', 600000.00),
    ('Fortnite World Cup 2024', 6,'2024-07-26','2024-07-28','New York, USA',       'Solo
Qualifiers',   3000000.00),
```

```
        ('BLAST Premier World Final 2024', 2,'2024-12-11','2024-12-15','Abu Dhabi,
UAE',      'Single Elimination',1000000.00),
    ('VCT Champions 2024', 1,'2024-08-01','2024-08-25','Seoul, South Korea', 'Double
Elimination', 500000.00);


-- INSERT: TOURNAMENT_REGISTRATION
INSERT INTO TOURNAMENT_REGISTRATION (tournament_id, team_id, registration_date, status)
VALUES
    (1,10,'2024-05-01','Confirmed'), (1,9,'2024-05-02','Confirmed'),
    (1,8,'2024-05-03','Confirmed'),  (1,11,'2024-05-04','Confirmed'),
    (2,2,'2024-03-01','Confirmed'),  (2,5,'2024-03-02','Confirmed'),
    (2,6,'2024-03-03','Confirmed'),  (2,12,'2024-03-04','Confirmed'),
    (3,2,'2024-06-20','Confirmed'),  (3,5,'2024-06-21','Confirmed'),
    (3,8,'2024-06-22','Confirmed'),  (3,6,'2024-06-23','Confirmed'),
    (4,7,'2024-09-01','Confirmed'),  (4,8,'2024-09-02','Confirmed'),
    (4,1,'2024-09-03','Confirmed'),  (4,4,'2024-09-04','Confirmed'),
    (5,1,'2024-07-20','Confirmed'),  (5,3,'2024-07-21','Confirmed'),
    (6,3,'2024-07-01','Confirmed'),  (6,6,'2024-07-02','Confirmed'),
    (7,2,'2024-11-15','Confirmed'),  (7,5,'2024-11-16','Confirmed'),
    (7,6,'2024-11-17','Confirmed'),  (7,12,'2024-11-18','Confirmed'),
    (8,10,'2024-07-10','Confirmed'), (8,9,'2024-07-11','Confirmed'),
    (8,8,'2024-07-12','Confirmed'),  (8,11,'2024-07-13','Confirmed');


-- INSERT: MATCH


INSERT INTO `MATCH` (tournament_id, team1_id, team2_id, match_date, stage, winner_team_id,
score) VALUES
    (1,10,9, '2024-06-10 14:00:00','Semifinal',10,'2-1'),
    (1,8,11, '2024-06-10 17:00:00','Semifinal', 8,'2-0'),
    (1,10,8, '2024-06-16 18:00:00','Final', 10,'3-1'),
    (2,2,12, '2024-04-01 13:00:00','Semifinal', 2,'2-0'),
    (2,5,6,  '2024-04-01 16:00:00','Semifinal', 5,'2-1'),
    (2,2,5,  '2024-04-07 17:00:00','Final', 5,'3-2'),
    (3,2,8,  '2024-07-18 13:00:00','Semifinal', 2,'2-1'),
    (3,5,6,  '2024-07-18 16:00:00','Semifinal', 6,'2-0'),
    (3,2,6,  '2024-07-21 17:00:00','Final', 2,'3-1'),
    (4,7,8,  '2024-10-30 14:00:00','Semifinal', 7,'3-1'),
    (4,1,4,  '2024-10-30 17:00:00','Semifinal', 1,'3-2'),
    (4,7,1,  '2024-11-02 18:00:00','Final', 7,'3-0'),
    (5,1,3,  '2024-08-10 15:00:00','Final', 1,'4-2'),
    (6,3,6,  '2024-07-28 16:00:00','Final', 6,'1-0'),
    (7,2,6,  '2024-12-13 14:00:00','Semifinal', 2,'2-0'),
    (7,5,12, '2024-12-13 17:00:00','Semifinal', 5,'2-1'),
    (7,2,5,  '2024-12-15 18:00:00','Final',      2,'3-2'),
    (8,10,8, '2024-08-22 14:00:00','Semifinal', 8,'2-1'),
    (8,9,11, '2024-08-22 17:00:00','Semifinal', 9,'2-0'),
    (8,8,9,  '2024-08-25 18:00:00','Final',      8,'3-2');
```

```
-- INSERT: PRIZE_DISTRIBUTION
INSERT INTO PRIZE_DISTRIBUTION (tournament_id, team_id, position, prize_amount) VALUES
    (1,10,1,200000.00),(1,8,2,140000.00), (1,9,3,80000.00),
    (2,5,1,350000.00),(2,2,2,200000.00), (2,6,3,100000.00),
    (3,2,1,400000.00),(3,6,2,250000.00), (3,5,3,125000.00),
    (4,7,1,900000.00),(4,1,2,450000.00), (4,8,3,225000.00),
    (5,1,1,300000.00),(5,3,2,150000.00),
    (6,6,1,1500000.00),(6,3,2,750000.00),
    (7,2,1,425000.00),(7,5,2,212500.00),(7,6,3,106250.00),
    (8,8,1,200000.00),(8,9,2,140000.00),(8,10,3,80000.00);

-- INSERT: SPONSOR
INSERT INTO SPONSOR (sponsor_name, industry, contact_email) VALUES
    ('Intel','Semiconductors','esports@intel.com'),
    ('Red Bull','Beverages','gaming@redbull.com'),
    ('SteelSeries','Gaming Peripherals','partners@steelseries.com'),
    ('ASUS ROG','Gaming Hardware','rog@asus.com'),
    ('HyperX','Gaming Peripherals', 'sponsorship@hyperx.com'),
    ('Secretlab','Gaming Furniture','deals@secretlab.co'),
    ('Monster Energy', 'Beverages','gaming@monsterenergy.com'),
    ('Mastercard','Financial Services', 'esports@mastercard.com'),
    ('BMW','Automotive','esports@bmw.com'),
    ('Logitech G','Gaming Peripherals','biz@logitechg.com');
-- INSERT: SPONSORSHIP
INSERT INTO SPONSORSHIP (sponsor_id, tournament_id, amount_contributed, contract_start,
contract_end) VALUES
    (1,1,80000.00,'2024-04-01','2024-06-30'),
    (2,1,60000.00,'2024-04-01','2024-06-30'),
    (3,1,40000.00,'2024-04-01','2024-06-30'),
    (4,2,200000.00,'2024-02-01','2024-04-30'),
    (5,2,150000.00,'2024-02-01','2024-04-30'),
    (6,2,80000.00,'2024-02-01','2024-04-30'),
    (1,3,300000.00,'2024-05-01','2024-07-31'),
    (7,3,200000.00,'2024-05-01','2024-07-31'),
    (8,3,150000.00,'2024-05-01','2024-07-31'),
    (9,4,500000.00,'2024-08-01','2024-11-30'),
    (8,4,400000.00,'2024-08-01','2024-11-30'),
    (2,4,300000.00,'2024-08-01','2024-11-30'),
    (10,5,150000.00,'2024-06-01','2024-08-31'),
    (4,5,100000.00,'2024-06-01','2024-08-31'),
    (2,6,800000.00,'2024-05-01','2024-08-31'),
    (9,6,600000.00,'2024-05-01','2024-08-31'),
    (7,7,250000.00,'2024-10-01','2024-12-31'),
    (5,7,180000.00,'2024-10-01','2024-12-31'),
    (6,7,120000.00,'2024-10-01','2024-12-31'),
    (3,8,90000.00,'2024-06-01','2024-08-31'),
    (10,8,70000.00,'2024-06-01','2024-08-31'),
    (1,8,50000.00,'2024-06-01','2024-08-31')
```

7.SQL QUERIES:

This section demonstrates various SQL operations performed on the database, including data manipulation and analytical queries used for generating meaningful reports to make sure that the project meets the required guidelines.

7.1 Data Insertion Queries

7.1.1 Insert a New Team

```
INSERT INTO TEAM (team_name, country)
VALUES ('Shadow Wolves', 'Europe');
```

7.1.2 Insert a New Player

```
INSERT INTO PLAYER (username, nationality )
VALUES ('AceGamer', 'USA');
```

7.1.3 Register a Team in a Tournament

```
INSERT INTO TOURNAMENT_REGISTRATION (team_id, tournament_id, registration_date)
VALUES (1, 2, '2026-04-01');
```

7.2 Data Update Queries

7.2.1 Update Team Region

```
UPDATE TEAM
SET country = 'Asia'
WHERE team_id = 1;
```

7.2.2 Update Prize Amount

```
UPDATE PRIZE_DISTRIBUTION
SET prize_amount = prize_amount + 5000
WHERE tournament_id = 2;
```

7.3 Data Deletion Queries

7.3.1 Delete a Player

```
DELETE FROM PLAYER
WHERE player_id = 10;
```

7.3.2 Remove Tournament Registration

```
DELETE FROM TOURNAMENT_REGISTRATION
WHERE team_id = 1 AND tournament_id = 2;
```

7.4 Analytical Queries (Reports)

7.4.1 Total Earnings of Each Team

```
SELECT t.team_name,
       SUM(pd.prize_amount) AS total_earnings
FROM TEAM t
LEFT JOIN PRIZE_DISTRIBUTION pd
ON t.team_id = pd.team_id
GROUP BY t.team_id, t.team_name
ORDER BY total_earnings DESC;
```

7.4.2 Top 3 Performing Teams

```
SELECT t.team_name,
       COUNT(m.match_id) AS wins
FROM `MATCH` m
JOIN TEAM t
ON m.winner_team_id = t.team_id
GROUP BY t.team_id, t.team_name
ORDER BY wins DESC
LIMIT 3;
```

7.4.3 Tournament with Highest Prize Pool

```
SELECT tournament_name, total_prize_pool
FROM TOURNAMENT
ORDER BY total_prize_pool DESC
LIMIT 1;
```

7.4.4 Total Sponsorship per Tournament

```
SELECT t.tournament_name,
       SUM(s.amount_contributed) AS total_sponsorship
FROM TOURNAMENT t
JOIN SPONSORSHIP s
ON t.tournament_id = s.tournament_id
GROUP BY t.tournament_id, t.tournament_name;
```

7.4.5 Player Earnings Leaderboard

```
SELECT *
FROM vw_player_earnings
ORDER BY total_earnings DESC;
```

7.4.6 Teams with No Earnings

```
SELECT t.team_name
FROM TEAM t
LEFT JOIN PRIZE_DISTRIBUTION pd
ON t.team_id = pd.team_id
WHERE pd.team_id IS NULL;
```

7.4.7 Most Popular Game (by Number of Tournaments)

```
SELECT g.game_name,
       COUNT(t.tournament_id) AS total_tournaments
FROM GAME g
LEFT JOIN TOURNAMENT t
ON g.game_id = t.game_id
GROUP BY g.game_id, g.game_name
ORDER BY total_tournaments DESC
LIMIT 1;
```

8. Database Optimization Report

8.1 Indexing Strategy

Indexes were created on foreign key columns that appear frequently in JOIN and WHERE clauses. The three indexes below directly accelerate the most common analytical queries:

Index Name	Table / Column	Benefit
idx_prize_team	PRIZE_DISTRIBUTION(team_id)	Accelerates per-team earnings lookups and vw_player_earnings
idx_prize_tournament	PRIZE_DISTRIBUTION(tournament_id)	Speeds up prize queries filtered by tournament
idx_sponsor_tourn	SPONSORSHIP(tournament_id)	Optimises sponsor-per-tournament JOIN operations

8.2 Triggers for Constraint Enforcement

MySQL does not support correlated subqueries inside CHECK constraints. The rule that a match winner must be one of the two competing teams was therefore enforced through BEFORE INSERT and BEFORE UPDATE triggers (trg_match_winner_insert and trg_match_winner_update). Both triggers are required — INSERT covers new rows; UPDATE covers score corrections.

8.3 Views for Derived Data

Rather than storing derived aggregates (total earnings, prize money) as physical columns — which would create update anomalies and 3NF violations — three views encapsulate the computation:

View	Purpose
vw_player_earnings	Traces PLAYER → ROSTER → PRIZE_DISTRIBUTION to compute total earnings per player
vw_team_earnings	Joins TEAM with PRIZE_DISTRIBUTION to compute total prize money per team
vw_global_leaderboard	Ranks all teams by total earnings and podium finish count for the global leaderboard

8.4 Normalization Benefits

Normalizing to 3NF produced the following measurable improvements to data integrity and maintainability:

- Elimination of update anomalies: prize additions no longer require manual column updates in PLAYER and TEAM.
- Elimination of insertion anomalies: a team can exist in the database without any prize record.
- Elimination of deletion anomalies: removing a prize record does not accidentally delete team data.
- Reduced storage footprint: aggregates are computed at query time rather than duplicated across rows.

9. Conclusion

The NexusArena project demonstrates a complete database development lifecycle applied to the esports domain. Beginning with entity–relationship modelling, the design was refined through systematic 3NF normalization, removing two derived columns and correcting three constraint weaknesses identified during review.

The implemented MySQL schema comprises ten tables, three analytical views, two trigger-enforced constraints, and three performance indexes. Sample data covering eight major 2024 esports tournaments — including VCT Masters Tokyo, IEM Cologne, and the LoL World Championship — validates the schema against real-world data volumes and relationships.

References

- Liquipedia — https://liquipedia.net
- HLTV.org — CS2 Statistics — https://www.hltv.org
- VLR.gg — Valorant Statistics — https://www.vlr.gg
- GOL.gg — League of Legends Statistics — https://gol.gg
- Esports Charts — Prize Pool Data — https://escharts.com
- Esports Insider — Sponsorship Data — https://esportsinsider.com
- draw.io — ER Diagram Design — https://app.diagrams.net

GUI APPLICATION CONNECTED TO DATABASE:

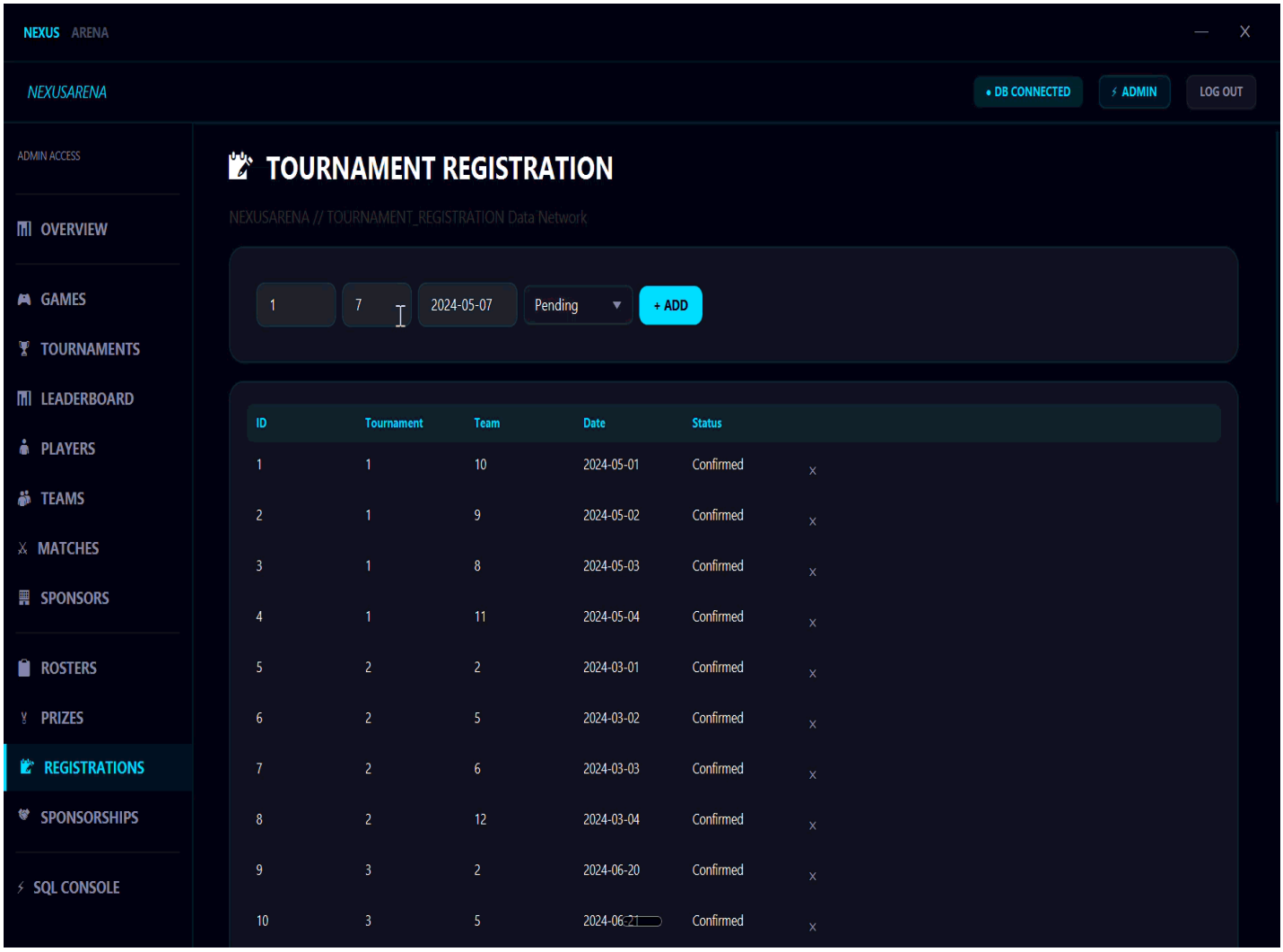
The team has also designed a GUI Application using the JavaFX framework . This application provides a easy approach to handle the database especially for someone not having expertise in the SQL.Our Application provides different Interface to the Different Users The Admin has Privilieges that a Player can not Have and similarly a player can do stuff that Community can not can.

- **Starting Screen:**

  The Screen Below shows the Loading page of the App and three different approaches possible in the case of our database.These three approaches in clude the Player,coomunity and Admin approach .

- **Admin Portal :**

  The Admin Portal provides the admin access to do changes in all the 10 tables included in the project The Application makes it easier for the admin to add and delete rows from the Tables without facing a need to write long SQL queries for such simple tasks

  SQL CONSOLE:

  This Feature of the app allows us to Run any SQL query Involving the retrieval of data in some way to be handled seamlessly.This feature makes life easier for the user so he or she won't have to switch screens Between The Application and the MySQL Workbench Again and Again.Complete control using one single app.